

Data Preprocessing: Needs for Preprocessing the Data, Data Cleaning, Data Integration and Transformation, Data Reduction, Discretization and Concept Hierarchy Generation.

Data preprocessing:

Data preprocessing describes any type of processing performed on raw data to prepare it for another processing procedure. Commonly used as a preliminary data mining practice, data preprocessing transforms the data into a format that will be more easily and effectively processed for the purpose of the user.

Why Data Preprocessing?

Data in the real world is dirty. It can be incomplete, noisy and inconsistent. These data needs to be preprocessed in order to help improve the quality of the data, and quality of the mining results.

- If no quality data, then no quality mining results. The quality decision is always based on the quality data.
- If there is much irrelevant and redundant information present or noisy and unreliable data, then knowledge discovery during the training phase is more difficult

Incomplete data: lacking attribute values, lacking certain attributes of interest, or containing only aggregate data. e.g., occupation=" ".

Noisy data: containing errors or outliers data. e.g., Salary="-10"

Inconsistent data: containing discrepancies in codes or names. e.g., Age="42" Birthday="03/07/1997"

Incomplete data may come from

- "Not applicable" data value when collected
- Different considerations between the time when the data was collected and when it is analysed.
- Human/hardware/software problems

Noisy data (incorrect values) may come from

- Faulty data collection by instruments
- Human or computer error at data entry
- Errors in data transmission

Inconsistent data may come from

- Different data sources
- Functional dependency violation (e.g., modify some linked data)

Major Tasks in Data Preprocessing:

1. Data cleaning
 - Fill in missing values, smooth noisy data, identify or remove outliers, and resolve inconsistencies
2. Data integration
 - Integration of multiple databases, data cubes, or files
3. Data reduction
 - Obtains reduced representation in volume but produces the same or similar analytical results
4. Data transformation
 - Normalization and aggregation
5. Data discretization
 - Part of data reduction but with particular importance, especially for numerical data

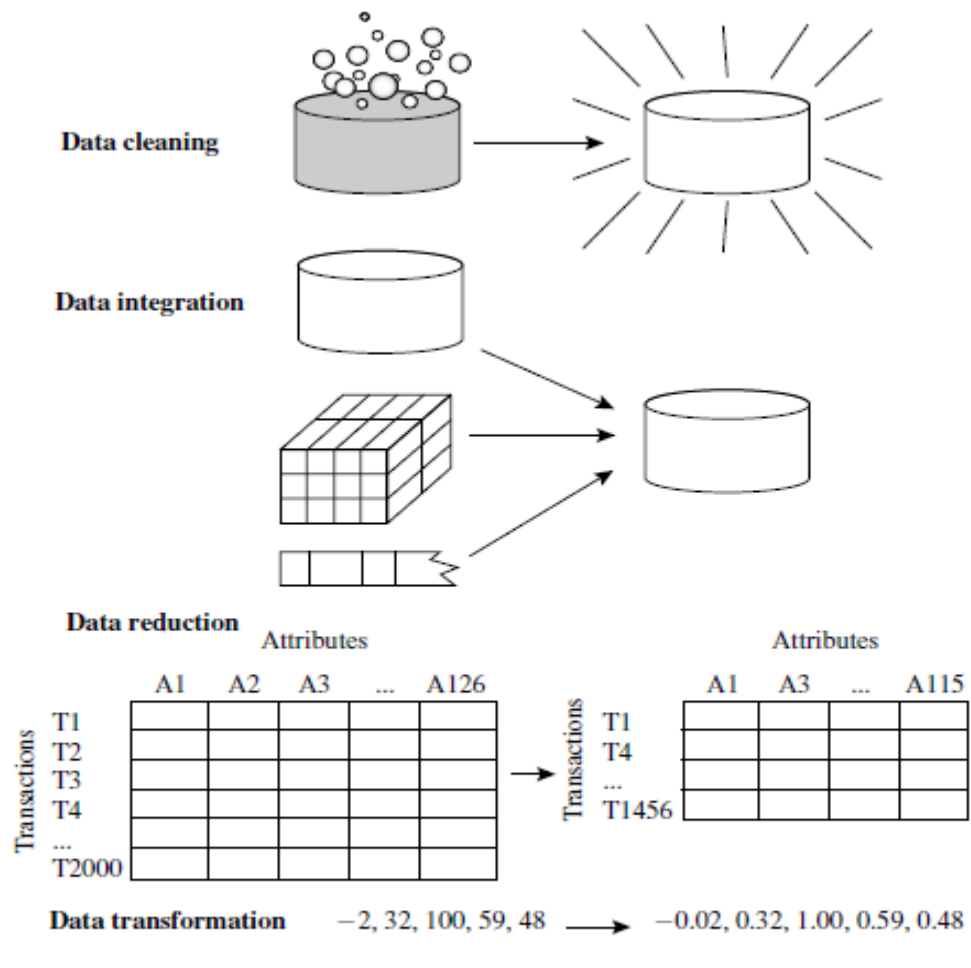


Figure 3.1 Forms of data preprocessing.

1. Data Cleaning

Data cleaning routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies in the data.

Various methods for handling this problem:

1.1 Missing Values

The various methods for handling the problem of missing values in data tuples include:

(a) Ignoring the tuple: This is usually done when the class label is missing (assuming the mining task involves classification or description). This method is not very effective unless the tuple contains several attributes with missing values. It is especially poor when the percentage of missing values per attribute varies considerably.

(b) Manually filling in the missing value: In general, this approach is time-consuming and may not be a reasonable task for large data sets with many missing values, especially when the value to be filled in is not easily determined.

(c) Using a global constant to fill in the missing value: Replace all missing attribute values by the same constant, such as a label like “Unknown,” or $-\infty$. If missing values are replaced by, say, “Unknown,” then the mining program may mistakenly think that they form an interesting concept, since they all have a value in common — that of “Unknown.” Hence, although this method is simple, it is not recommended.

(d) Using the attribute mean for quantitative (numeric) values or attribute mode for categorical (nominal) values, for all samples belonging to the same class as the given tuple: For example, if classifying customers according to credit risk, replace the missing value with the average income value for customers in the same credit risk category as that of the given tuple.

(e) Using the most probable value to fill in the missing value: This may be determined with regression, inference-based tools using Bayesian formalism, or decision tree induction. For example, using the other customer attributes in your data set, you may construct a decision tree to predict the missing values for income.

1.2 Noisy data:

Noise is a random error or variance in a measured variable. Data smoothing tech is used for removing such noisy data.

Several Data smoothing techniques:

1. Binning methods: Binning methods smooth a sorted data value by consulting the neighborhood", or values around it. The sorted values are distributed into a number of 'buckets', or bins. Because binning methods consult the neighborhood of values, they perform local smoothing.

In this technique,

1. The data for first sorted
2. Then the sorted list partitioned into equi-depth of bins.
3. Then one can smooth by bin means, smooth by bin median, smooth by bin boundaries, etc.
 - a. **Smoothing by bin means:** Each value in the bin is replaced by the mean value of the bin.
 - b. **Smoothing by bin medians:** Each value in the bin is replaced by the bin median.

c. **Smoothing by boundaries:** The min and max values of a bin are identified as the bin boundaries. Each bin value is replaced by the closest boundary value.

Example 1: Binning Methods for Data Smoothing

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- Partition into (equi-depth) bins (equi depth of 4 since each bin contains four values):

Bin 1: 4, 8, 9, 15

Bin 2: 21, 21, 24, 25

Bin 3: 26, 28, 29, 34

i. Smoothing by bin means:

Bin 1: 9, 9, 9, 9

Bin 2: 23, 23, 23, 23

Bin 3: 29, 29, 29, 29

In smoothing by bin means, each value in a bin is replaced by the mean value of the bin. For example, the mean of the values 4, 8, and 15 in Bin 1 is 9. Therefore, each original value in this bin is replaced by the value 9.

Similarly, smoothing by bin medians can be employed, in which each bin value is replaced by the bin median.

ii. Smoothing by bin Median:

median of numbers is the middle number when the numbers are written in order. If is even, the median is the average of the two middle numbers.

Ex: The marks of nine students in a geography test that had a maximum possible mark of 50 are given below:

47 35 37 32 38 39 36 34 35

Find the median of this set of data values.

Solution 1:

Arrange the data values in order from the lowest value to the highest value:

32 34 35 35 **36** 37 38 39 47

The fifth data value, 36, is the middle value in this arrangement. Hence, Median= 36.

Solution 2:

Median = $\frac{1}{2}(n+1)$ th value, where n is the number of data values in the sample

If the number of values in the data set is even, then the **median** is the average of the two middle values.

The number of values, n , in the data set = 9

$$\begin{aligned}\text{Median} &= \frac{1}{2}(9+1) \text{ th value} \\ &= 5\text{th value} \\ &= 36\end{aligned}$$

iii. Smoothing by bin boundaries:

Bin 1: 4, 4, 4, 15

Bin 2: 21, 21, 25, 25

Bin 3: 26, 26, 26, 34

In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Example 2: Suppose that the data for analysis include the attribute age. The age values for the data tuples are (in increasing order): 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70.

(a) Use smoothing by bin means to smooth the above data, using a bin depth of 3. Illustrate your steps.

Comment on the effect of this technique for the given data.

Solution: The following steps are required to smooth the above data using smoothing by bin means with a bin depth of 3.

- **Step 1:** Sort the data. (This step is not required here as the data are already sorted.)

- **Step 2:** Partition the data into equi-depth bins of depth 3.

Bin 1: 13, 15, 16

Bin 4: 22, 25, 25

Bin 7: 35, 35, 35

Bin 2: 16, 19, 20

Bin 5: 25, 25, 30

Bin 8: 36, 40, 45

Bin 3: 20, 21, 22

Bin 6: 33, 33, 35

Bin 9: 46, 52, 70

- **Step 3:** Calculate the arithmetic mean of each bin.

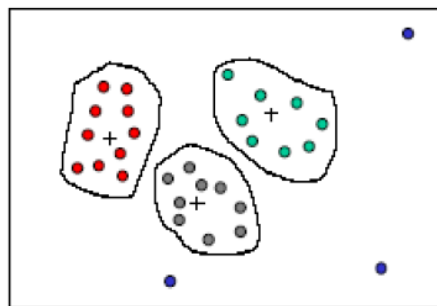
- **Step 4:** Replace each of the values in each bin by the arithmetic mean calculated for the bin.

Bin 1: 14, 14, 14 Bin 2: 18, 18, 18 Bin 3: 21, 21, 21

Bin 4: 24, 24, 24 Bin 5: 26, 26, 26 Bin 6: 33, 33, 33

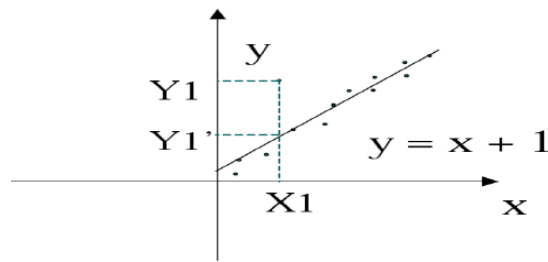
Bin 7: 35, 35, 35 Bin 8: 40, 40, 40 Bin 9: 56, 56, 56

2. **Clustering:** Outliers in the data may be detected by clustering, where similar values are organized into groups, or ‘clusters’. Values that fall outside of the set of clusters may be considered outliers.



3. **Regression:** smooth by fitting the data into regression functions.

- **Linear regression** involves finding the best of line to fit two variables, so that one variable can be used to predict the other.



- **Multiple linear regression** is an extension of linear regression, where more than two variables are involved and the data are fit to a multidimensional surface.

Using regression to find a mathematical equation to fit the data helps smooth out the noise.

Field overloading: is a kind of source of errors that typically occurs when developers compress new attribute definitions into unused portions of already defined attributes.

Unique rule is a rule says that each value of the given attribute must be different from all other values of that attribute

Consecutive rule is a rule says that there can be no missing values between the lowest and highest values of the attribute and that all values must also be unique.

Null rule specifies the use of blanks, question marks, special characters or other strings that may indicate the null condition and how such values should be handled.

2. Data Integration

It combines data from multiple sources into a coherent store. There are number of issues to consider during data integration.

Issues:

- **Schema integration:** refers integration of metadata from different sources.
- **Entity identification problem:** Identifying entity in one data source similar to entity in another table. For example, customer_id in one db and customer_no in another db refer to the same entity
- **Detecting and resolving data value conflicts:** Attribute values from different sources can be different due to different representations, different scales. E.g. metric vs. British units
- **Redundancy:** An attribute may be redundant if it can be “derived” from another attribute or set of attributes. Inconsistencies in attribute or dimension naming can also cause redundancies in the resulting data set.

Handling redundant data in data integration:

1. Correlation analysis

For numeric data:

Some redundancy can be identified by correlation analysis. The correlation between two variables A and B can be measured by

$$r_{A,B} = \frac{\sum_{i=1}^N (a_i - \bar{A})(b_i - \bar{B})}{N\sigma_A\sigma_B} = \frac{\sum_{i=1}^N (a_i b_i) - N\bar{A}\bar{B}}{N\sigma_A\sigma_B} \dots\dots (1)$$

where N is the number of tuples, a_i and b_i are the respective values of A and B in tuple i

$\bar{A}$ and $\bar{B}$ are the respective mean values of A and B

σ_A and σ_B are the respective standard deviations of A and B

Correlation range is $-1 \leq r_{A,B} \leq +1$.

- If $r_{A,B}$ is greater than 0, then A and B are positively correlated, meaning that the values of A increase as the values of B increase. The higher the value, the stronger the correlation.
- If the resulting value is equal to 0, then A and B are independent and there is no correlation between them.
- If the resulting value is less than 0, then A and B are negatively correlated.

For nominal data: a correlation relationship between two attributes, A and B, can be discovered by χ^2 (chi-square) test. Suppose A has c distinct values, namely $a_1, a_2, \dots, a_c$. B has r distinct values, namely $b_1, b_2, \dots, b_r$. The χ^2 value is computed as:

$$\chi^2 = \sum \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}} \quad (\text{or}) \quad \chi^2 = \sum_{i=1}^c \sum_{j=1}^r \frac{(o_{ij} - e_{ij})^2}{e_{ij}} \dots\dots (2)$$

where o_{ij} is the observed frequency of the joint event (A_i, B_j) and e_{ij} is the expected frequency of (A_i, B_j) , which can be computed as

$$e_{ij} = \frac{\text{count}(A = a_i) \times \text{count}(B = b_j)}{N} \dots\dots (3)$$

where N is the number of data tuples, $\text{count}(A=a_i)$ is the number of tuples having value a_i for A, and $\text{count}(B = b_j)$ is the number of tuples having value b_j for B. The sum is computed over all of the $r \times c$ cells.

The χ^2 statistic tests the hypothesis that A and B are independent. The test is based on a significance level, with $(r-1) \times (c-1)$ degrees of freedom. If the hypothesis can be rejected, then we say that A and B are statistically related or associated.

Example: Correlation analysis of nominal attributes using χ^2 :

Suppose that a group of 1500 people was surveyed. The gender of each person was noted. Each person was polled as to whether his or her preferred type of reading material was fiction or nonfiction. Thus, we have two attributes, gender and preferred reading.

Table 2.2 A 2×2 contingency table for the data of Example 2.1.
Are *gender* and *preferred_Reading* correlated?

	<i>male</i>	<i>female</i>	Total
<i>fiction</i>	250 (90)	200 (360)	450
<i>non_fiction</i>	50 (210)	1000 (840)	1050
Total	300	1200	1500

The observed frequency (or count) of each possible joint event is summarized in the contingency table shown in Table 2.2, where the numbers in parentheses are the expected frequencies. The expected frequencies are calculated based on the data distribution for both attributes using Eq. (3).

Using Eq. (3), we can verify the expected frequencies for each cell. For example, the expected frequency for the cell (male, fiction) is

$$e_{11} = \frac{\text{count}(\text{male}) \times \text{count}(\text{fiction})}{n} = \frac{300 \times 450}{1500} = 90$$

Using Eq. (2) for χ^2 computation, we get

$$\begin{aligned} \chi^2 &= \frac{(250 - 90)^2}{90} + \frac{(50 - 210)^2}{210} + \frac{(200 - 360)^2}{360} + \frac{(1000 - 840)^2}{840} \\ &= 284.44 + 121.90 + 71.11 + 30.48 = 507.93. \end{aligned}$$

For this 2X2 table, the degrees of freedom are $(2-1)(2-1)=1$. For 1 degree of freedom, the χ^2 value needed to reject the hypothesis at the 0.001 significance level is 10.828. Hence, gender and preferred reading are independent and not correlated.

4. Data Transformation

The data are transformed or consolidated into forms appropriate for mining. Data transformation can involve the following:

- **Smoothing:** which works to remove noise from the data
- **Aggregation:** where summary or aggregation operations are applied to the data. For example, the daily sales data may be aggregated so as to compute weekly and annual total scores.
- **Generalization of the data:** where low-level or “primitive” (raw) data are replaced by higher-level concepts through the use of concept hierarchies. For example, categorical attributes, like street, can be generalized to higher-level concepts, like city or country.
- **Normalization:** where the attribute data are scaled so as to fall within a small specified range, such as -1.0 to 1.0 , or 0.0 to 1.0 .
- **Attribute construction (feature construction):** this is where new attributes are constructed and added from the given set of attributes to help the mining process.

Normalization: An attribute is normalized by scaling its values so that they fall within a small specified range, such as 0.0 to 1.0. Normalization is useful for classification algorithms involving neural networks, or distance measurements such as nearest-neighbour classification and clustering.

There are several methods for data normalization. In that three methods are:

- 1) min-max normalization
- 2) z-score normalization
- 3) Normalization by decimal scaling

1) Min-max normalization: It performs a linear transformation on the original data. Suppose that $\min_A$ and $\max_A$ are the minimum and maximum values of an attribute, A. Min-max normalization maps a value, v, of A to v' in the range $[\text{new } \min_A, \text{new } \max_A]$ by computing

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

It will encounter an “out-of-bounds” error if a future input case for normalization falls outside of the original data range for A.

Ex: Suppose that the minimum and maximum values for the attribute income are \$12,000 and \$98,000, respectively. We would like to map income to the range [0.0, 1.0]. By min-max normalization, a value of \$73,600 for income is transformed to

$$\frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0) + 0 = 0.716.$$

2) Z-score normalization / zero-mean normalization: In which values of an attribute A are normalized based on the mean and standard deviation of A. A value v, of A is normalized to v' by computing

$$v' = \frac{v - \text{mean}_A}{\text{stand_dev}_A}$$

This method is useful when min and max value of attribute A are unknown or when outliers that are dominate min-max normalization.

Ex: Suppose that the mean and standard deviation of the values for the attribute income are \$54,000 and \$16,000, respectively. With z-score normalization, a value of \$73,600 for income is transformed to

$$\frac{73,600 - 54,000}{16,000} = 1.225.$$

3) Normalization by decimal scaling: normalizes by moving the decimal point of values of attribute A. The number of decimal points moved depends on the maximum absolute value of A. A value v of A is normalized to v' by computing,

$$v' = \frac{v}{10^j} \quad \text{Where } j \text{ is the smallest integer such that } \text{Max}(|v'|) < 1$$

Ex: Suppose that the recorded values of A range from -986 to 917. The maximum absolute value of A is 986. To normalize by decimal scaling, we divide each value by 1,000 (i.e., $j = 3$) so that -986 normalizes to -0.986 and 917 normalizes to 0.917.

3. Data Reduction techniques

These techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data. Strategies for Data reduction include the following:

1. **Data cube aggregation**, where aggregation operations are applied to the data in the construction of a data cube.
2. **Attribute subset selection**, where irrelevant, weakly relevant or redundant attributes or dimensions may be detected and removed.
3. **Dimensionality reduction**, where encoding mechanisms are used to reduce the data set size. Examples: Wavelet Transforms Principal Components Analysis
4. **Numerosity reduction**, where the data are replaced or estimated by alternative, smaller data representations such as parametric models (which need store only the model parameters instead of the actual data) or nonparametric methods such as clustering, sampling, and the use of histograms.
5. **Discretization and concept hierarchy generation**, where raw data values for attributes are replaced by ranges or higher conceptual levels. Data Discretization is a form of numerosity reduction that is very useful for the automatic generation of concept hierarchies.

1. Data cube aggregation:

Data cubes store multidimensional aggregated information. Queries regarding aggregated information should be answered using data cube when possible. The figure shows a data cube for multidimensional analysis of sales data with respect to annual sales per item type for each AllElectronics branch. These data consist of the sales per quarter, for the years 2002 to 2004.

Year 2004	
Quarter	Sales
Year 2003	0
	0
	0
	0
Year 2002	0
	0
	0
	0
Quarter	Sales
Q1	\$224,000
Q2	\$408,000
Q3	\$350,000
Q4	\$586,000



Year	Sales
2002	\$1,568,000
2003	\$2,356,000
2004	\$3,594,000

Figure 2.13 Sales data for a given branch of AllElectronics for the years 2002 to 2004. On the left, the sales are shown per quarter. On the right, the data are aggregated to provide the annual sales.

Suppose, the analyzer interested in the annual sales rather than sales per quarter, the data can be aggregated so that the resulting data summarize the total sales per year instead of per quarter. This aggregation is illustrated in Figure. The resulting data set is smaller in volume, without loss of information necessary for the analysis task. Each cell holds an aggregate data value, corresponding to the data point in multidimensional space.

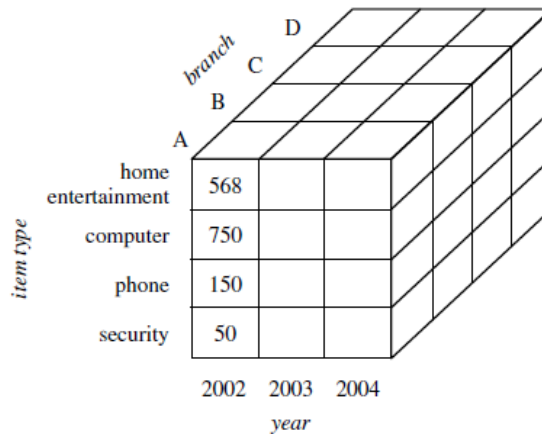


Figure 2.14 A data cube for sales at AllElectronics.

- Data cubes provide fast access to pre-computed, summarized data, thereby benefiting on-line analytical processing as well as data mining.
- The cube created at the lowest level of abstraction is referred to as the base cuboid.
- A cube for the highest level of abstraction is the apex cuboid.
- The lowest level of a data cube (base cuboid). Data cubes created for varying levels of abstraction are sometimes referred to as cuboids.
- Each higher level of abstraction further reduces the resulting data size.

2. Attribute sub selection / Feature selection

Feature selection is a must for any data mining product. because, when you build a data mining model, the dataset frequently contains more information than is needed to build the model. For example, a dataset may contain 500 columns that describe characteristics of customers, but perhaps only 50 of those columns are used to build a particular model. If you keep the unneeded columns while building the model, more CPU and memory are required during the training process, and more storage space is required for the completed model.

- Attribute subset selection reduces the data set size by removing irrelevant or redundant attributes (or dimensions).
- The goal of attribute subset selection is to find a minimum set of attributes.
- Mining on a reduced set of attributes has an additional benefit.
- It reduces the number of attributes appearing in the discovered patterns, helping to make the patterns easier to understand.

Basic **heuristic methods** of attribute subset selection include the following techniques and illustrated in Figure:

- 1. Step-wise forward selection:** The procedure starts with an empty set of attributes. The best of the original attributes is determined and added to the set.
- 2. Step-wise backward elimination:** The procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.
- 3. Combination forward selection and backward elimination:** The step-wise forward selection and backward elimination methods can be combined, where at each step one selects the best attribute and removes the worst from among the remaining attributes.
- 4. Decision tree induction:**
 - Decision tree induction constructs a flow-chart-like structure where each internal (non-leaf) node denotes a test on an attribute, each branch corresponds to an outcome of the test, and each external (leaf) node denotes a class prediction.
 - At each node, the algorithm chooses the “best” attribute to partition the data into individual classes.
 - A tree is constructed from the given data.
 - All attributes that do not appear in the tree are assumed to be irrelevant. The set of attributes appearing in the tree form the reduced subset of attributes.

Forward selection	Backward elimination	Decision tree induction
Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ Initial reduced set: $\{\}$ $\Rightarrow \{A_1\}$ $\Rightarrow \{A_1, A_4\}$ $\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ $\Rightarrow \{A_1, A_3, A_4, A_5, A_6\}$ $\Rightarrow \{A_1, A_4, A_5, A_6\}$ $\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ <pre> graph TD A4["A4?"] -- Y --> A1["A1?"] A4 -- N --> A6["A6?"] A1 -- Y --> C1_1("Class 1") A1 -- N --> C2_1("Class 2") A6 -- Y --> C1_2("Class 1") A6 -- N --> C2_2("Class 2") </pre> $\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$

Figure 2.15 Greedy (heuristic) methods for attribute subset selection.

2. Dimensionality Reduction

In dimensionality reduction, data encoding or transformations are applied so as to obtain a reduced or “compressed” representation of the original data. If the original data can be reconstructed from the compressed data without any loss of information, the data reduction is called lossless. The two popular and effective methods of lossy dimensionality reduction are:

- a. Wavelet transforms
- b. Principal components analysis

a. Wavelet compression is a form of data compression well suited for image compression. The discrete wavelet transform (DWT) is a linear signal processing technique, when it applied to a data vector D , transforms it to a numerically different vector, D_0 , of wavelet coefficients.

- Haar2 and Daubechie4 are two popular wavelet transforms.

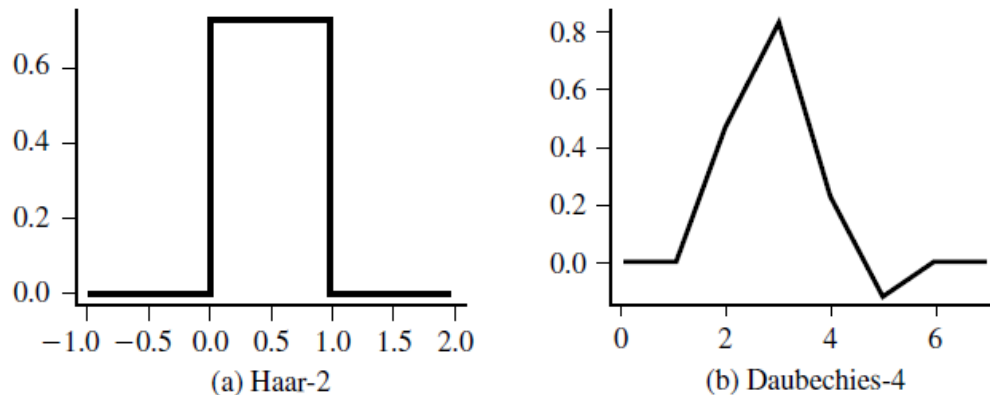


Figure 2.16 Examples of wavelet families. The number next to a wavelet name is the number of *vanishing moments* of the wavelet. This is a set of mathematical relationships that the coefficients must satisfy and is related to the number of coefficients.

The general procedure for applying a discrete wavelet transform uses a hierarchical pyramid algorithm resulting in fast computational speed. The method is as follows:

1. The length, L , of the input data vector must be an integer power of two. This condition can be met by padding the data vector with zeros, as necessary.
2. Each transform involves applying two functions:
 - data smoothing
 - calculating weighted difference
3. The two functions are applied to pairs of the input data, resulting in two sets of data of length $L/2$.
4. The two functions are recursively applied to the sets of data obtained in the previous loop, until the resulting data sets obtained are of desired length.
5. A selection of values from the data sets obtained in the above iterations are designated the wavelet coefficients of the transformed data.

b. Principal Component Analysis (PCA):

Suppose that the data to be reduced consist of tuples or data vectors described by n attributes or dimensions. Principal components analysis (PCA) also called as Karhunen-Loeve (K-L) method. It searches for k n -dimensional orthogonal vectors that can best be used to represent the data, where $k \leq n$. The original data are thus projected onto a much smaller space, resulting in dimensionality reduction.

- Works for ordered and unordered attributes
- Used when the number of dimensions is large

The basic procedure is as follows:

1. The input data are normalized, so that each attribute falls within the same range.
2. PCA computes k orthonormal vectors that provide a basis for the normalized input data. These vectors are referred to as the principal components. The input data are a linear combination of the principal components.
3. The principal components are sorted in order of decreasing “significance” or strength. The principal components essentially serve as a new set of axes for the data, providing important information about variance.

For **example**, Figure 2.17 shows the first two principal components, Y_1 and Y_2 , for the given set of data originally mapped to the axes X_1 and X_2 . This information helps identify groups or patterns within the data.

4. Because the components are sorted according to decreasing order of “significance,” the size of the data can be reduced by eliminating the weaker components, that is, those with low variance. The strongest components are used to reconstruct a good approximation of the original signal.

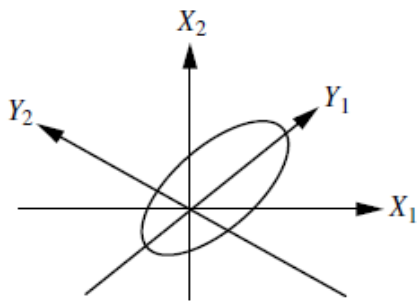


Figure 2.17 Principal components analysis. Y_1 and Y_2 are the first two principal components for the given data.

4. Numerosity Reduction

Data volume can be reduced by choosing alternative smaller forms of data representation. This techniques may be

- Parametric method
- Non parametric method

Parametric: A model is used to estimate the data, only the data parameters need to be stored, instead of the actual data.

Non parametric: includes histogram, clustering and sampling methods for storing reduced representations of the data.

Numerosity reduction techniques:

1. Regression and Log-Linear Models: Can be used to approximate the given data

- **Linear regression**, the data are modelled to fit a straight line using

$Y = \alpha + \beta X$, where α, β are coefficients

- **Multiple linear regression:** it is an extension of linear regression, which allows a response variable, y ,

to be modelled as a linear function of two or more predictor variables.

$$Y = b_0 + b_1 X_1 + b_2 X_2.$$

Log-linear model: approximate discrete multidimensional probability distributions. These models are also useful for dimensionality reduction and data smoothing.

$$\text{Probability: } p(a, b, c, d) = \alpha_{ab} \beta_{ac} \gamma_{ad} \delta_{bcd}$$

2. Histograms

Histogram for an attribute 'A' partitions the data distribution of A into disjoint subsets or buckets. If each bucket represents only a single attribute-value/frequency pair, the buckets are called singleton buckets. Often, buckets represent continuous ranges for the given attribute.

Example: Histograms. The following data are a list of prices of commonly sold items at All Electronics (rounded to the nearest dollar). The numbers have been sorted: 1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30. Draw histogram plot for price where each bucket should have equi-width of 10

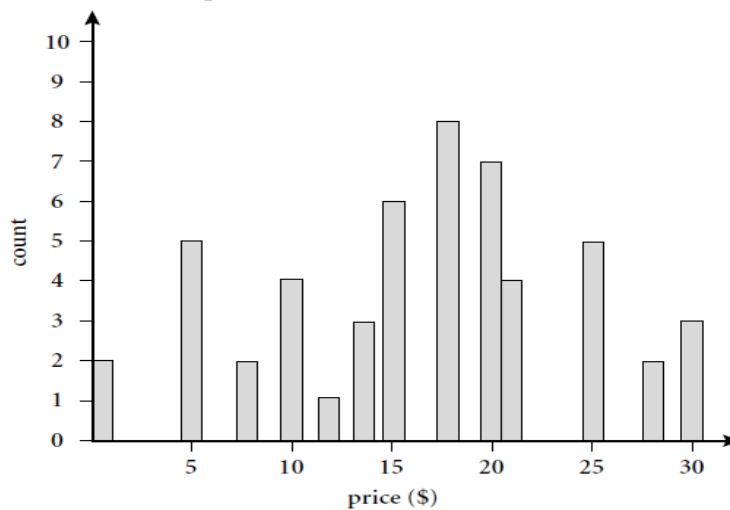


Figure 2.18 A histogram for *price* using singleton buckets—each bucket represents one price-value/frequency pair.

Figure 2.18 shows a histogram for the data using singleton buckets. To further reduce the data, it is common to have each bucket denote a continuous range of values for the given attribute.

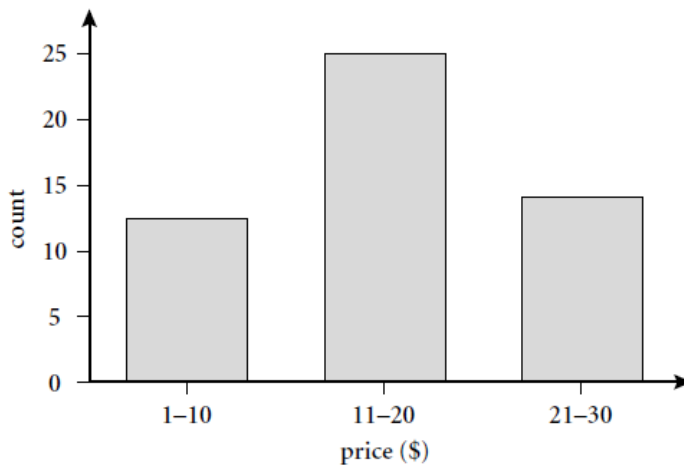


Figure 2.19 An equal-width histogram for *price*, where values are aggregated so that each bucket has a uniform width of \$10.

In Figure 2.19, each bucket represents a different \$10 range for price.

The buckets can be determined based on the following partitioning rules:

1. Equi-width: the width of each bucket range is uniform.
2. Equi-depth: the frequency of each bucket is constant
3. V-Optimal: histogram with least variance
4. MaxDiff: considers the difference between each pair of adjacent values. A bucket boundary is established between each pair for pairs having the $\beta - 1$ largest differences, where β is the user-specified number of buckets.

Clustering: Clustering techniques consider data tuples as objects. They partition the objects into groups or clusters, so that objects within a cluster are “similar” to one another and “dissimilar” to objects in other clusters. Centroid distance is an alternative measure of cluster quality and is defined as the average distance of each cluster object from the cluster centroid. The cluster representations of the data are used to replace the actual data.

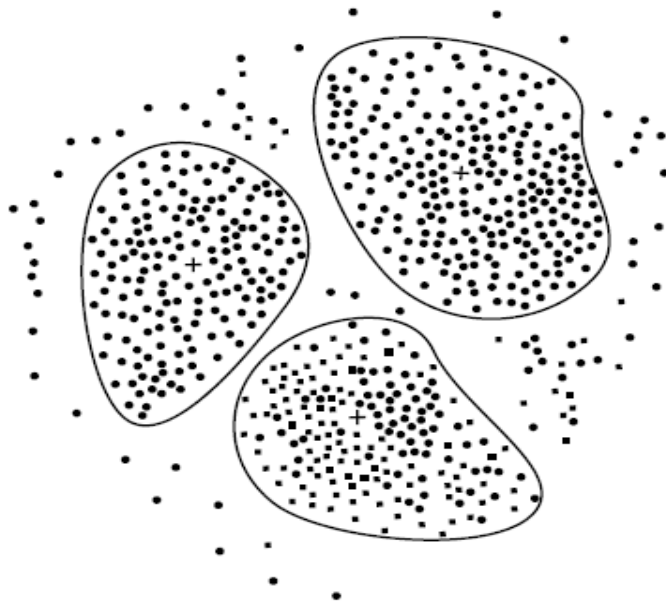


Figure 2.12 A 2-D plot of customer data with respect to customer locations in a city, showing three data clusters. Each cluster centroid is marked with a “+”, representing the average point in space for that cluster. Outliers may be detected as values that fall outside of the sets of clusters.

Sampling: Sampling can be used as a data reduction technique as it allows a large data set to be represented by a much smaller random sample (or subset) of the data. Suppose that a large data set D contains N tuples. Let's have a look at the ways for data reduction illustrated in fig.

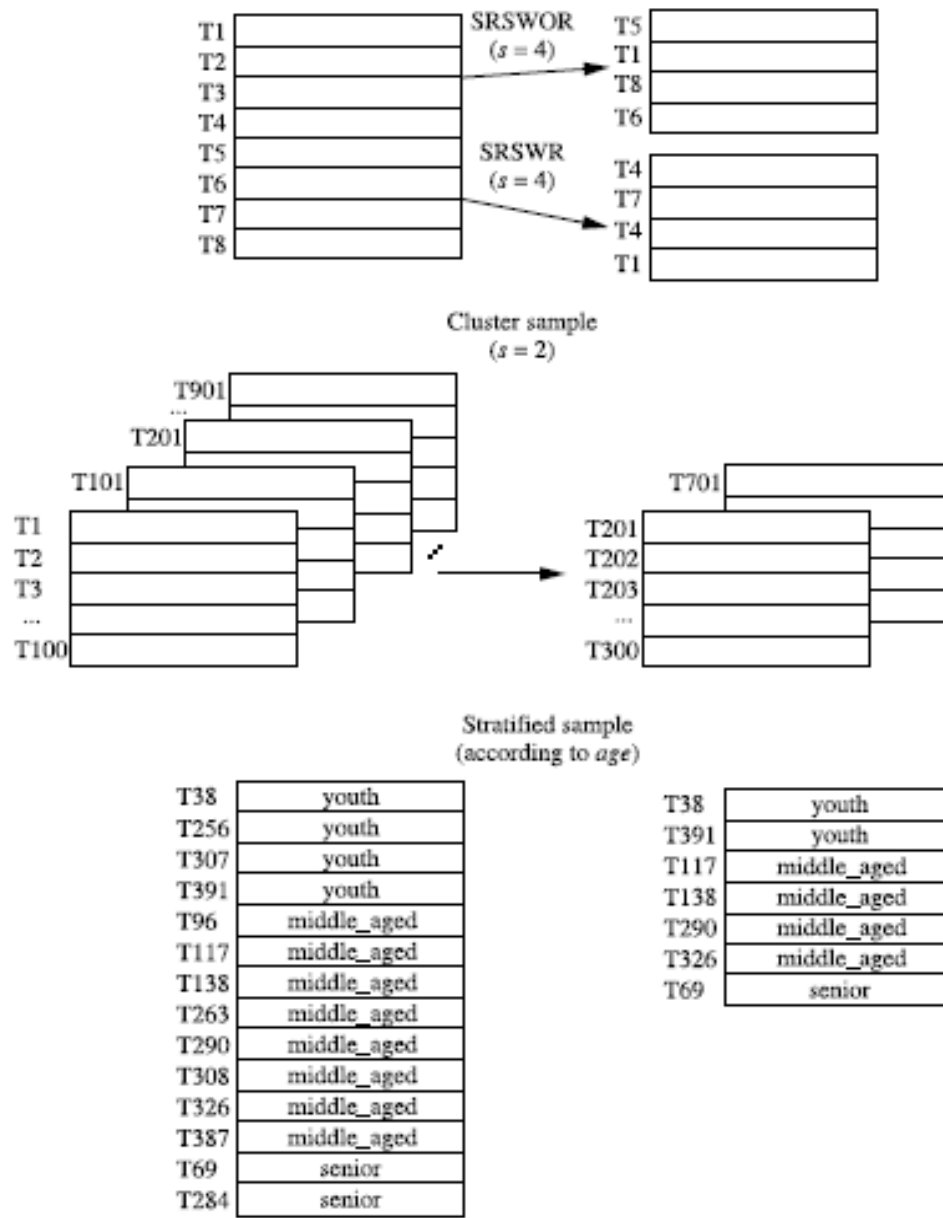


Figure 2.21 Sampling can be used for data reduction.

- 1. Simple random sample without replacement (SRSWOR) of size n :** This is created by drawing n of the N tuples from D ($n < N$), where the probability of drawing any tuple in D is $1/N$, i.e., all tuples are equally likely.
- 2. Simple random sample with replacement (SRSWR) of size n :** This is similar to SRSWOR, except that each time a tuple is drawn from D , it is recorded and then replaced.
- 3. Cluster sample:** If the tuples in D are grouped into M mutually disjoint "clusters", then a SRS of m clusters can be obtained, where $m < M$.
- 4. Stratified sample:** If D is divided into mutually disjoint parts called "strata", a stratified sample of D is generated by obtaining a SRS at each stratum. This helps to ensure a representative sample, especially when the data are skewed.

Advantages of sampling:

1. An advantage of sampling for data reduction is that the cost of obtaining a sample is proportional to the size of the sample, n , as opposed to N , the data set size. Hence, sampling complexity is potentially sub-linear to the size of the data.
2. When applied to data reduction, sampling is most commonly used to estimate the answer to an aggregate query.

5. Data Discretization and Concept Hierarchy Generation

Discretization:

- Discretization techniques can be used to reduce the number of values for a given continuous attribute, by dividing the range of the attribute into intervals.
- Interval labels can then be used to replace actual data values. thereby reduces and simplifies the original data.
- Discretization techniques can be categorized based on how the discretization is performed, such as whether it uses class information or which direction it proceeds (i.e., top-down vs. bottom-up).
- If the discretization process uses class information, then we say it is **supervised** discretization. Otherwise, it is **unsupervised**.
- If the process starts by first finding one or a few points (called split points or cut points) to split the entire attribute range, and then repeats this recursively on the resulting intervals, it is called **top-down** discretization or splitting.
- This contrasts with **bottom-up** discretization or merging, which starts by considering all of the continuous values as potential split-points, removes some by merging neighborhood values to form intervals, and then recursively applies this process to the resulting intervals.
- Discretization can be performed recursively on an attribute to provide a hierarchical or multi-resolution partitioning of the attribute values, known as a concept hierarchy.
- Concept hierarchies are useful for mining at multiple levels of abstraction.

Concept Hierarchy:

A concept hierarchy for a given numeric attribute defines a Discretization of the attribute. Concept hierarchies can be used to reduce the data by collecting and replacing low level concepts (such as numeric values for the attribute age) by higher level concepts (such as young, middle-aged, or senior).

Ex: An example of a concept hierarchy for the attribute price is given in Figure 2.22. More than one concept hierarchy can be defined for the same attribute in order to accommodate the needs of various users.

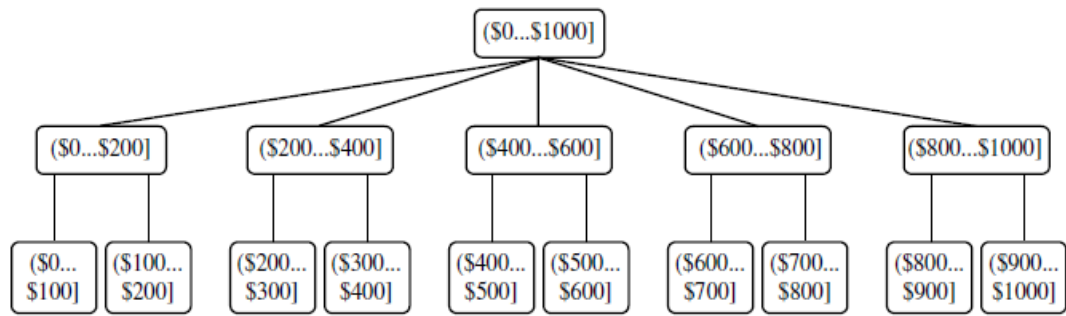


Figure 2.22 A concept hierarchy for the attribute *price*, where an interval $(\$X \dots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive).

i) Discretization and Concept Hierarchy Generation for Numerical data

Concept hierarchies for numerical attributes can be constructed automatically based on data discretization. the methods are: binning, histogram analysis, entropy-based discretization, χ^2 -merging, cluster analysis, and discretization by intuitive partitioning. each method assumes that the values to be discretized are sorted in ascending order.

Binning:

- Binning is a top-down splitting technique based on a specified number of bins. It also used as discretization methods for numerosity reduction and concept hierarchy generation.
- For example, attribute values can be discretized by applying equal-width or equal-frequency binning, and then replacing each bin value by the bin mean or median, as in smoothing by bin means or smoothing by bin medians, respectively.
- These techniques can be applied recursively to the resulting partitions in order to generate concept hierarchies. Binning does not use class information and is therefore an unsupervised discretization technique.

Histogram Analysis:

It is an unsupervised discretization technique as it does not use class information.

Histograms partition the values for an attribute 'A' into disjoint ranges called buckets.

Partitioning rules are used for defining histograms.

This algorithm can be applied recursively to each partition in order to automatically generate a multilevel concept hierarchy.

A minimum interval size can also be used per level to control the recursive procedure. This specifies the minimum width of a partition, or the minimum number of values for each partition at each level.

Entropy-Based Discretization:

- Entropy is one of the most commonly used discretization measures.
- Entropy-based discretization is a supervised, top-down splitting technique.

- It explores class distribution information in its calculation and determination of split-points (data values for partitioning an attribute range).
- To discretize a numerical attribute 'A' the method selects the value of A that has the minimum entropy as a split-point, and recursively partitions the resulting intervals to arrive at a hierarchical discretization. Such discretization forms a concept hierarchy for A.
- Let D consist of data tuples defined by a set of attributes and a class-label attribute. The class-label attribute provides the class information per tuple.

The basic method for entropy-based discretization of an attribute A within the set is as follows:

1. Each value of A can be considered as a potential interval boundary or split-point to partition the range of A. i.e. a split-point for A can partition the tuples in D into two subsets satisfying the conditions $A \leq \text{split point}$ and $A > \text{split point}$ thereby creating a binary discretization.
 2. It uses information regarding the class label of tuples. Suppose we want to classify the tuples in D by partitioning on attribute A and some split-point. Ideally, we would like this partitioning to result in an exact classification of the tuples.
 3. The process of determining a split-point is recursively applied to each partition obtained, until some stopping criterion is met.
- Entropy-based discretization can reduce data size.
 - It uses class information to define the interval boundaries (split-points) in places that may help improve classification accuracy.

Interval Merging by χ^2 Analysis:

ChiMerge is a χ^2 -based discretization method. It employs a bottom-up approach by finding the best neighboring intervals and then merging these to form larger intervals, recursively.

The method is supervised and it uses class information.

The basic notion is that for accurate discretization, the relative class frequencies should be fairly consistent within an interval. Therefore, if two adjacent intervals have a very similar distribution of classes, then the intervals can be merged. Otherwise, they should remain separate.

ChiMerge proceeds as follows:

- Initially, each distinct value of a numerical attribute A is considered to be one interval.
- χ^2 tests are performed for every pair of adjacent intervals.
- Adjacent intervals with the least χ^2 values are merged together, because low χ^2 values for a pair indicate similar class distributions.
- This merging process proceeds recursively until a predefined stopping criterion is met.

The stopping criterion is typically determined by three conditions.

- First, merging stops when c_2 values of all pairs of adjacent intervals exceed some threshold, which is determined by a specified significance level. The significance level is set between 0.10 and 0.01.
- Second, the number of intervals cannot be over a prespecified max-interval, such as 10 to 15.
- In practice, some inconsistency is allowed, although this should be no more than a prespecified threshold, such as 3%, which may be estimated from the training data. This last condition can be used to remove irrelevant attributes from the data set.

Cluster Analysis:

- Cluster analysis is a popular data discretization method.
- A clustering algorithm can be applied to discretize a numerical attribute A by partitioning the values of A into clusters or groups.
- Clustering can be used to generate a concept hierarchy for A by following either a top-down splitting strategy or a bottom-up merging strategy, where each cluster forms a node of the concept hierarchy.
- Each initial cluster or partition may be further decomposed into several sub-clusters, forming a lower level of the hierarchy.
- Clusters are formed by repeatedly grouping neighboring clusters in order to form higher-level concepts.

Discretization by Intuitive Partitioning:

The numerical ranges are partitioned into relatively uniform, easy-to-read intervals that appear intuitive or natural.”

The 3-4-5 rule can be used to segment numerical data into relatively uniform, natural seeming intervals.

The rule partitions a given range of data into 3, 4, or 5 relatively equal-width intervals, recursively and level by level, based on the value range at the most significant digit.

The rule is as follows:

- If an interval covers 3, 6, 7, or 9 distinct values at the most significant digit, then partition the range into 3 intervals (3 equal-width intervals for 3, 6, and 9; and 3 intervals in the grouping of 2-3-2 for 7).
- If it covers 2, 4, or 8 distinct values at the most significant digit, then partition the range into 4 equal-width intervals.
- If it covers 1, 5, or 10 distinct values at the most significant digit, then partition the range into 5 equal-width intervals.

The rule can be recursively applied to each interval, creating a concept hierarchy for the given numerical attribute.

Ex: Numeric concept hierarchy generation by intuitive partitioning. Suppose that profits at different branches of AllElectronics for the year 2004 cover a wide range, from -\$351,976.00 to

\$4,700,896.50. A user desires the automatic generation of a concept hierarchy for profit. For improved readability, we use the notation $[l..r]$ to represent the interval $[l, r]$. For example, $(-\$1,000,000...\$0]$ denotes the range from $-\$1,000,000$ (exclusive) to $\$0$ (inclusive). Suppose that the data within the 5th percentile and 95th percentile are between $-\$159,876$ and $\$1,838,761$. The results of applying the 3-4-5 rule are shown in Figure 2.23.

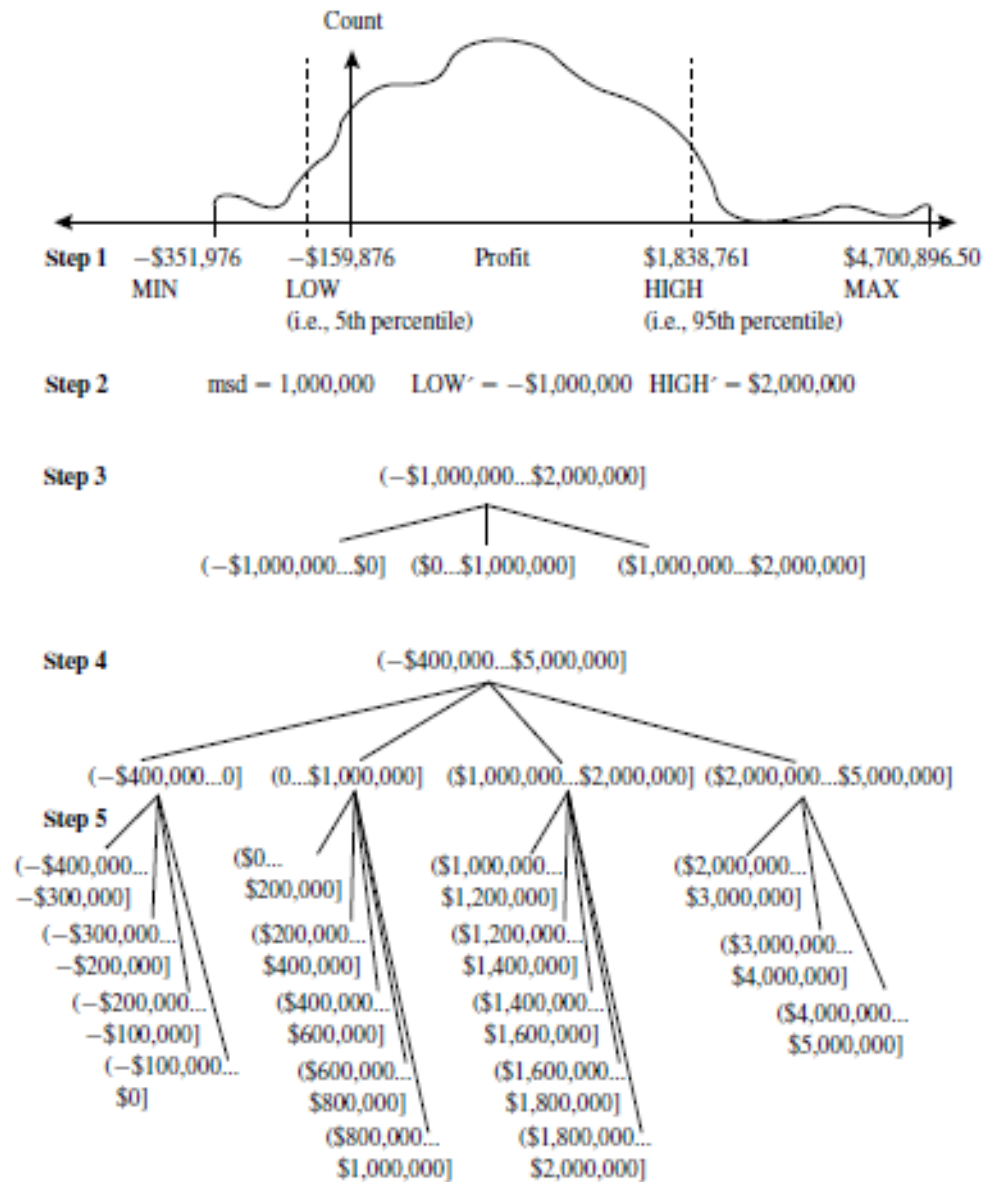


Figure 2.23 Automatic generation of a concept hierarchy for *profit* based on the 3-4-5 rule.

1. Based on the above information, the minimum and maximum values are MIN = $-\$351,976.00$, and MAX = $\$4,700,896.50$. The low (5th percentile) and high (95th percentile) values to be considered for the top or first level of discretization are LOW = $-\$159,876$, and HIGH = $\$1,838,761$.
2. Given LOW and HIGH, the most significant digit (msd) is at the million dollar digit position (i.e., msd = 1,000,000). Rounding LOW down to the million dollar digit, we get LOW0 = $-\$1,000,000$; rounding HIGH up to the million dollar digit, we get HIGH0 = $+\$2,000,000$.

3. Since this interval ranges over three distinct values at the most significant digit, that is, $(2,000,000 - (-1,000,000)) = 1,000,000 = 3$, the segment is partitioned into three equal-width sub-segments according to the 3-4-5 rule: $(-1,000,000 \dots 0]$, $(0 \dots 1,000,000]$, and $(1,000,000 \dots 2,000,000]$. This represents the top tier of the hierarchy.
4. We now examine the MIN and MAX values to see how they “fit” into the first-level partitions. Since the first interval $(-1,000,000 \dots 0]$ covers the MIN value, that is, $LOW_0 < MIN$, we can adjust the left boundary of this interval to make the interval smaller. The most significant digit of MIN is the hundred thousand digit position. Rounding MIN down to this position, we get $MIN_0 = -\$400,000$. Therefore, the first interval is redefined as $(-400,000 \dots 0]$. Since the last interval, $(1,000,000 \dots 2,000,000]$, does not cover the MAX value, that is, $MAX > HIGH_0$, we need to create a new interval to cover it. Rounding up MAX at its most significant digit position, the new interval is $(2,000,000 \dots 5,000,000]$. Hence, the topmost level of the hierarchy contains four partitions, $(-400,000 \dots 0]$, $(0 \dots 1,000,000]$, $(1,000,000 \dots 2,000,000]$, and $(2,000,000 \dots 5,000,000]$.
5. Recursively, each interval can be further partitioned according to the 3-4-5 rule to form the next lower level of the hierarchy:
 - The first interval, $(-400,000 \dots 0]$, is partitioned into 4 subintervals: $(-400,000 \dots -300,000]$, $(-300,000 \dots -200,000]$, $(-200,000 \dots -100,000]$, and $(-100,000 \dots 0]$.
 - The second interval, $(0 \dots 1,000,000]$, is partitioned into 5 subintervals: $(0 \dots 200,000]$, $(200,000 \dots 400,000]$, $(400,000 \dots 600,000]$, $(600,000 \dots 800,000]$, and $(800,000 \dots 1,000,000]$.
 - The third interval, $(1,000,000 \dots 2,000,000]$, is partitioned into 5 subintervals: $(1,000,000 \dots 1,200,000]$, $(1,200,000 \dots 1,400,000]$, $(1,400,000 \dots 1,600,000]$, $(1,600,000 \dots 1,800,000]$, and $(1,800,000 \dots 2,000,000]$.
 - The last interval, $(2,000,000 \dots 5,000,000]$, is partitioned into 3 subintervals: $(2,000,000 \dots 3,000,000]$, $(3,000,000 \dots 4,000,000]$, and $(4,000,000 \dots 5,000,000]$.

Similarly, the 3-4-5 rule can be carried on iteratively at deeper levels, as necessary

ii) Concept hierarchy generation for categorical data

Categorical data are discrete data. Categorical attributes have a finite number of distinct values, with no ordering among the values.

Ex: geographic location, job category, and item type.

There are several **methods** for the generation of concept hierarchies for categorical data.

- **Specification of a partial ordering of attributes explicitly at the schema level by users or experts:**

Concept hierarchies for categorical attributes or dimensions typically involve a group of attributes. A user or expert can easily define a concept hierarchy by specifying a partial or total ordering of the attributes at the schema level.

- **Specification of a portion of a hierarchy by explicit data grouping:**

This is a manual definition of a portion of a concept hierarchy. It defines an entire concept hierarchy by explicit value enumeration.

- **Specification of a set of attributes, but not of their partial ordering:**

A user may specify a set of attributes forming a concept hierarchy, but omit to explicitly state their partial ordering. The system can then try to automatically generate the attribute ordering to construct a meaningful concept hierarchy.

Ex: Concept hierarchy generation based on the number of distinct values per attribute.

Suppose a user selects a set of location-oriented attributes, street, country, province or state, and city, from the AllElectronics database, but does not specify the hierarchical ordering among the attributes. A concept hierarchy for location can be generated automatically as illustrated in Figure 2.24.

- First, sort the attributes in ascending order based on the number of distinct values in each attribute. This results in the following (where the number of distinct values per attribute is shown in parentheses): country (15), province or state (365), city (3567), and street (674,339).
- Second, generate the hierarchy from the top down according to the sorted order, with the first attribute at the top level and the last attribute at the bottom level.
- Finally, the user can examine the generated hierarchy, and when necessary, modify it to reflect desired semantic relationships among the attributes.
- In this example, it is obvious that there is no need to modify the generated hierarchy.

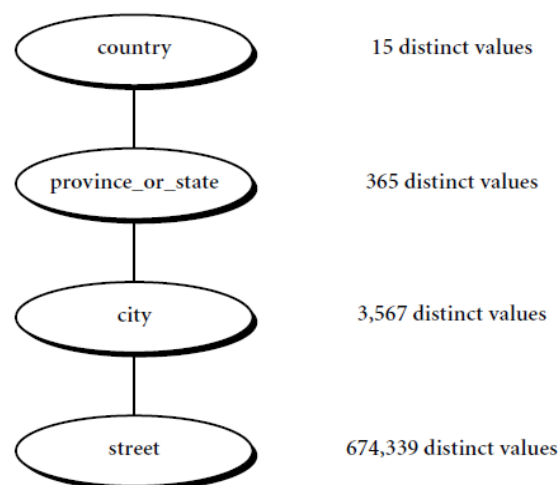


Figure 2.24 Automatic generation of a schema concept hierarchy based on the number of distinct attribute values.

Measures of Similarity and Dissimilarity:

- In data mining applications, such as clustering, outlier analysis, and nearest-neighbor classification, we need ways to assess alike or unlike objects. So how can we compare one object from another?
- **Ex:** a store may want to search for clusters of customer objects, resulting in groups of customers with similar characteristics (e.g., similar income, area of residence, and age).
- A cluster is a collection of data objects such that the objects within a cluster are similar to one another and dissimilar to the objects in other clusters.
- Outlier analysis also employs clustering-based techniques to identify potential outliers as objects that are highly dissimilar to others.
- Knowledge of object similarities can also be used in nearest-neighbor classification schemes where a given object (e.g., a patient) is assigned a class label (relating to, say, a diagnosis) based on its similarity toward other objects in the model.
- Similarity and dissimilarity measures are referred as measures of proximity.
- A similarity measure for two objects i and j return the value 1 which indicates objects are alike. The higher the similarity i.e. the objects are identical.
- A dissimilarity measure returns a value of 0 if the objects are unlike. The higher the dissimilarity value, the more dissimilar the two objects are.
- For this types of applications, two data structures are commonly used: data matrix (used to store the data objects) and the dissimilarity matrix (used to store dissimilarity values for pairs of objects).

Data Matrix versus Dissimilarity Matrix:

- Consider objects described by multiple attributes.
- Suppose that we have n objects (e.g., persons, items, or courses) described by p attributes called as measurements or features such as age, height, weight, or gender.
- The objects are $x_1 = (x_{11}, x_{12}, \dots, x_{1p})$, $x_2 = (x_{21}, x_{22}, \dots, x_{2p})$, and so on, where x_{ij} is the value for object x_i of the j^{th} attribute. Where object x_i referred as object i .
- The objects may be tuples in a relational database, and are also referred to as data samples or feature vectors.
- Main memory-based clustering and nearest-neighbor algorithms typically operate on either of the following two data structures:

Data matrix (or object-by-attribute structure): This structure stores the n data objects in the form of a relational table, or n -by- p matrix (n objects $\times$ p attributes)

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}$$

Each row corresponds to an object.

Dissimilarity matrix (or object-by-object structure): This structure stores a collection of proximities that are available for all pairs of n objects. It is often represented by an n-by-n table:

$$\begin{bmatrix} 0 & & & & \\ d(2, 1) & 0 & & & \\ d(3, 1) & d(3, 2) & 0 & & \\ \vdots & \vdots & \vdots & \ddots & \\ d(n, 1) & d(n, 2) & \dots & \dots & 0 \end{bmatrix}$$

where $d(i, j)$ is the measured dissimilarity or “difference” between objects i and j . In general, $d(i, j)$ is a non-negative number that is close to 0 when objects i and j are highly similar.

➤ Data mining primitives

The data mining primitives specify the following,

1. Set of task-relevant data to be mined.
2. Kind of knowledge to be mined.
3. Background knowledge to be used in the discovery process.
4. Interestingness measures and thresholds for pattern evaluation.
5. Representation for visualizing the discovered patterns.

➤ Data Mining Query Languages

The Data Mining Query Language is actually based on Structured Query Language (SQL). Data Mining Query Languages can be designed to support ad hoc and interactive data mining. This DMQL provides commands for specifying primitives.

Task-Relevant Data Specification Syntax

```
use database database_name,
or
use data warehouse data_warehouse_name
in relevance to att_or_dim_list
from relation(s)/cube(s) [where condition]
order by order_list
group by grouping_list
```

Specifying Kind of Knowledge Syntax

Here we will discuss the syntax for Characterization, Discrimination, Association, Classification and Prediction.

Characterization

The syntax for characterization is:

```
mine characteristics [as pattern_name]
  analyze {measure(s) }
```

The analyze clause, specifies aggregate measures, such as count, sum, or count%.

For example:

Description describing customer purchasing habits.

```
mine characteristics as customerPurchasing
analyze count%
```

Discrimination

The syntax for Discrimination is:

```
mine comparison [as {pattern_name}]
For {target_class } where {target_condition }
{versus {contrast_class_i }
where {contrast_condition_i}}
analyze {measure(s) }
```

Association

The syntax for Association is:

```
mine associations [ as {pattern_name} ]
{matching {metapattern} }
```

➤ **Architectures of Data Mining Systems**

Data mining is the process of discovering interesting knowledge from large amounts of data stored either in databases, data warehouses, or other information repositories.

Based on this view, the architecture of a typical data mining system may have the following major components:

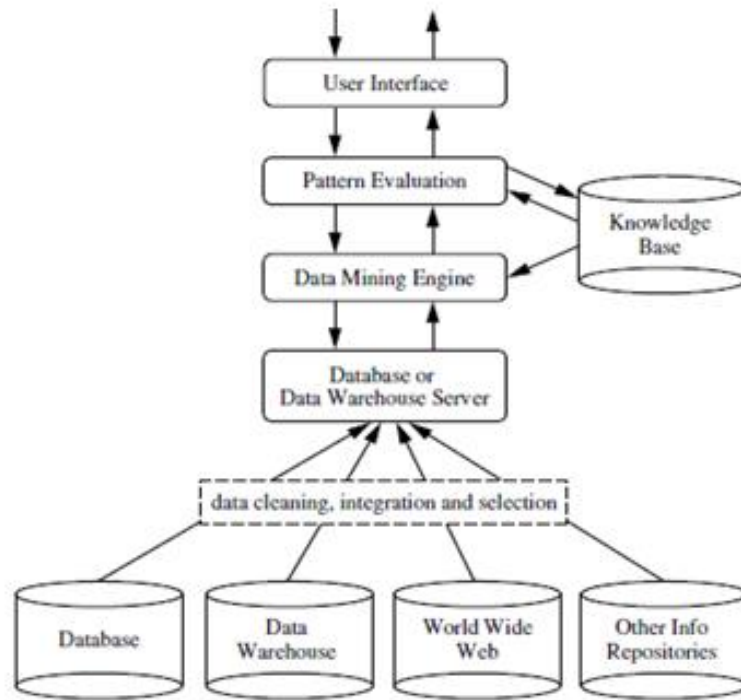


Fig. Architecture of a typical data mining system

1. A database, data warehouse, or other information repository, which consists of the set of databases, data warehouses, spreadsheets, or other kinds of information repositories containing the student and course information.
2. A database or data warehouse server which fetches the relevant data based on users data mining requests.
3. A knowledge base that contains the domain knowledge used to guide the search or to evaluate the interestingness of resulting patterns. For example, the knowledge base may contain metadata which describes data from multiple heterogeneous sources.
4. A data mining engine, which consists of a set of functional modules for tasks such as classification, association, classification, cluster analysis, and evolution and deviation analysis.
5. A pattern evaluation module that works with the data mining modules by employing interestingness measures to help focus the search towards interestingness patterns.
6. A graphical user interface that allows the user an interactive approach to the data mining system.

Interesting Measures:

A pattern is interesting if

- (1) It is easily understood by humans
- (2) Valid on new or test data with some degree of certainty
- (3) Potentially useful, and
- (4) Novel.